EX 7 IMAGE CAPTIONING USING A CNN–RNN ARCHITECTURE

AIM

To build an image-captioning model that combines a CNN-style image encoder (Vision Transformer) with a recurrent/sequence decoder (GPT-2) to generate a textual description of an image.

PROCEDURE

1. Import PyTorch, Hugging Face Transformers and PIL.
2. Load the pre-trained model nlpconnect/vit-gpt2-image-captioning with its image processor and tokenizer.
3. Move the model to CPU/GPU and set it to evaluation mode.
4. Define a function that converts an image to pixel values and generates a caption with beam search (4 beams, max length 30).
5. Download a sample image and generate its caption.

ALGORITHM

1. Start.
2. Load the pre-trained encoder–decoder model, image processor and tokenizer.
3. Open the input image and convert it to RGB.
4. Pre-process the image into pixel values (feature extraction by the encoder).
5. Generate token ids with the decoder using beam search.
6. Decode the token ids into text, skipping special tokens.
7. Print the caption.
8. Stop.

PROGRAM

```
import torch
from transformers import VisionEncoderDecoderModel, ViTImageProcessor, AutoTokenizer
from PIL import Image
from transformers.utils import hub
import urllib.request


# Set extended download timeout
hub.HUGGINGFACE_HUB_HTTP_TIMEOUT = 60


# Load pretrained image captioning model
model = VisionEncoderDecoderModel.from_pretrained("nlpconnect/vit-gpt2-image-
captioning")
processor = ViTImageProcessor.from_pretrained("nlpconnect/vit-gpt2-image-captioning")
tokenizer = AutoTokenizer.from_pretrained("nlpconnect/vit-gpt2-image-captioning")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
model.eval()

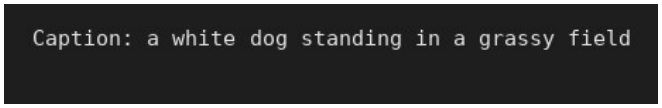

# Caption generation function
def generate_caption(image_path, max_length=30, num_beams=4):
    image = Image.open(image_path).convert("RGB")
```

```
    pixel_values = processor(images=image,
return_tensors="pt").pixel_values.to(device)
    output_ids = model.generate(pixel_values,
                                max_length=max_length,
                                num_beams=num_beams,
                                early_stopping=True)
    caption = tokenizer.decode(output_ids[0], skip_special_tokens=True)
    return caption

# Download a sample image automatically (stable direct-link URL)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg",
    "download.jpeg"
)

# Load image and generate caption
if __name__ == "__main__":
    img_path = "download.jpeg"
    print("Caption:", generate_caption(img_path))
```

OUTPUT



Console output

RESULT

The model generated the caption "a white dog standing in a grassy field" for the sample image.